



Collège de
Maisonneuve

SERVICE D'ÉCHANGE DES DONNÉES

00SV - DA 420-4D2-MA

Avril 2026

Andrea Silva

Table des matières

I – Introduction	4
Le flux principal de l'application	4
Les technologies utilisées	4
Objectifs de l'API.....	4
Authentification	5
Sécurité de l'API.....	5
II – Description des environnements	5
1. Description de l'environnement de déploiement.....	5
2. Architecture globale du système	6
3. Serveur	6
4. Infrastructure web	7
5. Description de l'environnement de base de données	7
6. Environnement d'exécution du paiement	7
7. Sécurité de l'environnement	8
III - Analyse	8
Étendue du projet	8
Fonctionnalités incluses :.....	8
Fonctionnalités exclues :.....	8
Spécifications Fonctionnelles.....	9
Flux fonctionnel global	9
Flux détaillé du paiement	11
Règles de gestion	14
IV – Conception.....	19
Architecture de l'API	19
Structure des endpoints	19
Headers http.....	21
Codes de réponse http	21
Sécurité de l'API.....	22
Diagrammes UML	24
Diagramme de classe.....	24
Diagramme de séquence.....	25

Plan de test (Contient des cas de test).....	26
Stratégie / approche de test.....	26
Exemples de cas de test	27
Endpoint testé :	30
V – Planification.....	33
VI – Développement	33
Structure du projet :	33
Description des composants	34
Controllers.....	34
Models.....	34
Views.....	34
Public	34
Intégration de l’API dans MVC	35
Fonctionnement.....	35
Gestion de l’authentification	37
Séparation Web / API.....	38
Structure de la base de données.....	38
VII – Déploiement	43
Plan de déploiement en production	43
Préparation de l’environnement :	43
Transfert des fichiers :	43
Configuration de l’API.....	43
Tests en production.....	44
Spécifications de l’hébergement AwardSpace.....	44
Sécurité en production	44
Vérification post-déploiement	45
Spécifications de l’hébergement.....	45
Conclusion	47

I – Introduction

Ce projet consiste à développer une API REST pour une application web de vente de matériel informatique intégrant un processus complet de transaction, allant de la gestion du panier jusqu'au paiement et à l'enregistrement des transactions.

L'API joue un rôle central dans l'application en assurant la communication entre le frontend et le backend, tout en encapsulant la logique métier. Elle permet de structurer les échanges de données et de faciliter l'évolution vers une architecture plus découplée et orientée services.

Le flux principal de l'application

Repose sur les étapes suivantes :

- ◆ Gestion du panier (ajout, modification, suppression)
- ◆ Traitement des paiements
- ◆ Enregistrement des transactions et gestion des expéditions

L'API est conçue pour centraliser la logique métier et offrir une structure claire pour le traitement des données.

Les technologies utilisées

Dans ce projet sont :

- ◆ Backend / API: PHP (architecture MVC)
- ◆ Frontend: HTML, CSS, Bootstrap, JavaScript
- ◆ Base de données : MySQL
- ◆ Hébergement : AwardSpace

Objectifs de l'API

- ◆ Centraliser la logique métier dans des endpoints REST
- ◆ Assurer une communication structurée entre frontend et backend
- ◆ Garantir la cohérence du flux transactionnel (panier → paiement → transaction)
- ◆ Sécuriser les opérations sensibles, notamment les paiements
- ◆ Faciliter la maintenance et l'évolution de l'application

Base URL de l'API :

- ◆ <http://etransactionnelle.atwebpages.com/api/>

Format des données :

- ◆ Toutes les requêtes et réponses de l'API sont au format JSON.

Authentication

L'accès aux endpoints protégés de l'API est sécurisé par un système de tokens.

En raison des limitations de l'hébergement mutualisé AwardSpace, le header standard Authorization: Bearer ne peut pas être exploité correctement côté serveur.

Une solution basée sur un token personnalisé a donc été implémentée.

Le token est généré lors de l'authentification, stocké en base de données et transmis via les requêtes HTTP dans un header dédié.

Sécurité de l'API

L'API intègre plusieurs mécanismes de sécurité afin de protéger les données et les opérations sensibles :

- ◆ Validation des données côté serveur (formats, champs obligatoires)
- ◆ Vérification des règles métier (stock, solde, cohérence des transactions)
- ◆ Utilisation de requêtes préparées pour prévenir les injections SQL
- ◆ Protection contre les attaques XSS
- ◆ Limitation des stockages des données sensibles (ex : stockage partiel des cartes bancaires)

Ces mécanismes permettent d'assurer la fiabilité et la sécurité du processus transactionnel.

II – Description des environnements

1. Description de l'environnement de déploiement

L'API est déployée sur un environnement d'hébergement mutualisé via AwardSpace.

Cet environnement permet d'exécuter des applications PHP et de gérer une base de données MySQL.

Le déploiement se fait via FTP (FileZilla), en transférant les fichiers vers le serveur distant.

L'API est accessible publiquement via des endpoints HTTP :

<http://etransactionnelle.atwebpages.com/api/>

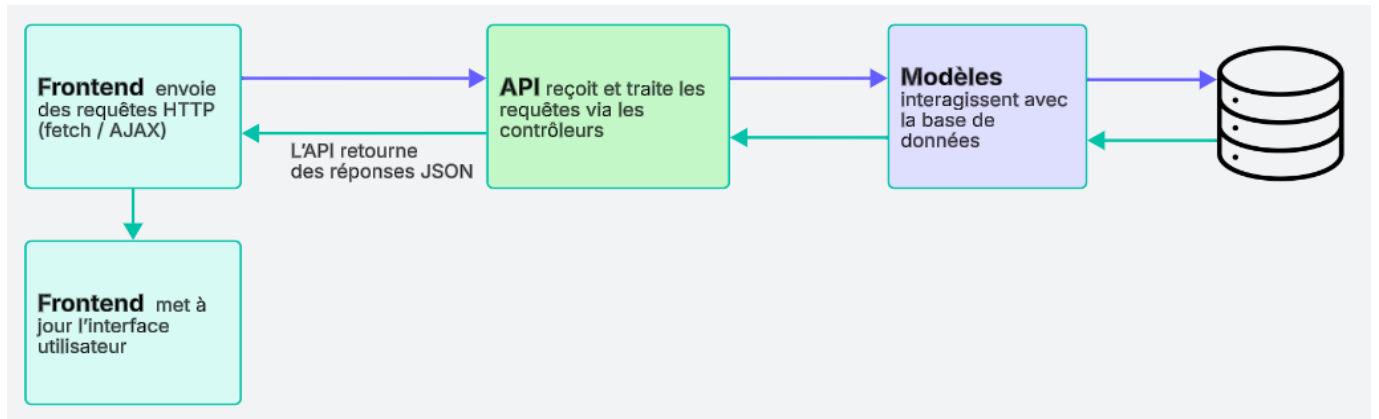
Cet environnement héberge à la fois :

1. Le backend (API PHP en architecture MVC)
2. Le frontend (interfaces HTML/CSS/JS)

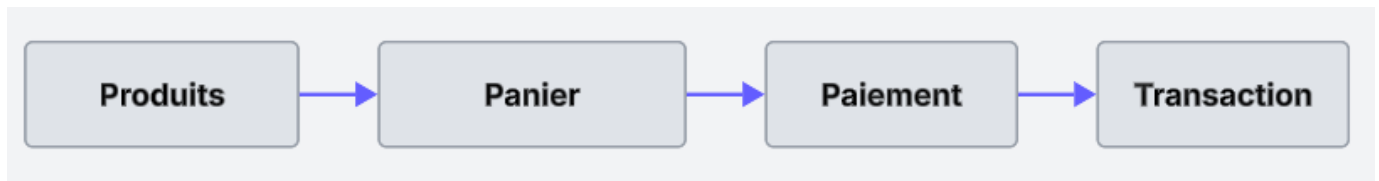
2. Architecture globale du système

L'application repose sur une architecture **client - serveur orientée API**.

Le fonctionnement est le suivant :



Le flux principal métier suit la séquence :



3. Serveur

Le serveur utilisé est un serveur web mutualisé qui supporte :

1. PHP (exécution des contrôleurs et logique métier)
2. Apache (gestion des requêtes HTTP et routing via .htaccess)
3. MySQL (stockage des données)

Le routage est assuré via un point d'entrée unique (index.php) qui redirige les requêtes vers les contrôleurs appropriés.

4. Infrastructure web

L'infrastructure repose sur une communication HTTP/HTTPS entre :

- ◆ Le client (navigateur)
- ◆ Le serveur web (Apache)
- ◆ L'API backend
- ◆ La base de données

Les échanges sont structurés en JSON pour assurer une communication standardisée.

Les endpoints principaux exposés sont :

- ◆ /api/login
- ◆ /api/cart
- ◆ /api/payment
- ◆ /api/expedition
- ◆ /api/logout

5. Description de l'environnement de base de données

L'application utilise une base de données relationnelle MySQL.

Rôle de la base de données :

- ◆ Stockage des utilisateurs
- ◆ Stockage des produits
- ◆ Gestion du panier (temporaire ou persistant)
- ◆ Enregistrement des transactions après paiement

Type de base de données :

- ◆ MySQL (compatible AwardSpace)

6. Environnement d'exécution du paiement

Le paiement est géré via un système simulé (PSP) intégré dans l'API.

Fonctionnement :

- ◆ L'API reçoit les informations de paiement
- ◆ Les données sont validées côté serveur
- ◆ Une simulation de transaction bancaire est effectuée
- ◆ En cas de succès :
 - ◆ La transaction est enregistrée
 - ◆ Le panier est validé (et éventuellement vidé)

Ce mécanisme permet de reproduire un flux réel sans dépendre d'un fournisseur externe.

7. Sécurité de l'environnement

Plusieurs mécanismes assurent la sécurité :

- ◆ Authentification via token (Bearer)
- ◆ Validation des entrées utilisateur
- ◆ Protection contre les injections SQL
- ◆ Protection contre les attaques XSS
- ◆ Contrôle des accès aux endpoints sensibles (paiement, transactions)
- ◆ Utilisation recommandée du protocole HTTPS afin de sécuriser les échanges entre le client et l'API

III - Analyse

Cette section présente l'analyse fonctionnelle de l'API.

Elle permet de définir les fonctionnalités principales, le comportement attendu et les règles de gestion.

L'API est conçue pour répondre aux besoins d'une application de vente en ligne avec gestion des utilisateurs, des produits, du panier et des paiements.

Étendue du projet

L'API couvre les fonctionnalités suivantes :

Fonctionnalités incluses :

- ◆ Consultation des produits
- ◆ Gestion du panier
- ◆ Traitement des paiements
- ◆ Gestion des transactions

Fonctionnalités exclues :

- ◆ Gestion avancée des rôles utilisateurs
- ◆ Intégration avec un système de paiement réel
- ◆ Gestion complète d'inventaire

Spécifications Fonctionnelles

Fonctionnalité	Description / Comportement attendu	Entrées	Sorties / Résultat
Produits	◆ Afficher les produits	-	Liste des produits
Panier	◆ Ajouter / supprimer produits	Id produit, quantité	Panier mis à jour
Paieement	◆ Traiter un paiement	Info paiement, montant	succès / erreur
Transactions	◆ Enregistrer et consulter	données transaction	historique

Flux fonctionnel global

Le fonctionnement principal de l'application repose sur un enchaînement d'étapes permettant de passer de la sélection de produits jusqu'à la validation d'une transaction.

Ce flux constitue le cœur métier du système et s'appuie sur plusieurs endpoints de l'API.

Étapes du flux :

1. Consultation des produits
L'utilisateur récupère la liste des produits via l'API (/api/products).
2. Ajout au panier
L'utilisateur ajoute un ou plusieurs produits au panier via (/api/cart).
Le panier est mis à jour dynamiquement.
3. Validation du panier
Avant le paiement, le système vérifie que :
 - ◆ Le panier contient au moins un produit
 - ◆ Les quantités sont valides

4. Envoi de la requête de paiement

Le frontend envoie une requête POST vers (/api/payment) contenant :

- ◆ Les informations de paiement
- ◆ Le montant total
- ◆ Les produits du panier (ou référence au panier)

5. Traitement du paiement

Le backend :

- ◆ Valide les données
- ◆ Simule une transaction bancaire (PSP simulé)
- ◆ Vérifie les conditions (ex: solde, validité)

6. Résultat du paiement

Deux cas possibles :

a. Succès

- ◆ Paiement accepté
- ◆ Création d'une transaction en base de données
- ◆ Réponse JSON avec statut "success"

b. Échec

- ◆ Paiement refusé
- ◆ Aucun enregistrement de transaction valide
- ◆ Retour d'un message d'erreur

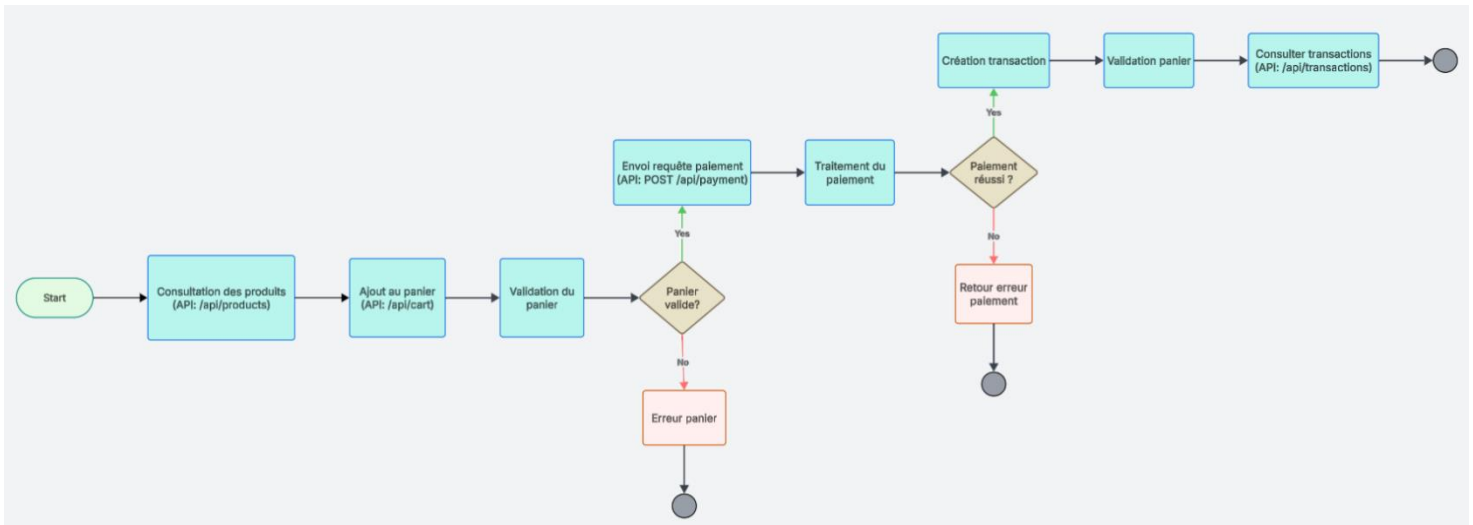
7. Finalisation

En cas de succès :

- ◆ Le panier est validé (et éventuellement vidé)
- ◆ L'utilisateur peut consulter ses transactions via (/api/transactions)

Ce flux garantit la cohérence entre les données du panier, le paiement et l'historique des transactions.

Diagramme de Flux Fonctionnel Global



Flux détaillé du paiement

Le processus de paiement est entièrement géré par l'endpoint API (/api/payment) et repose sur l'envoi des données complètes du panier, des informations de paiement et des informations d'expédition dans le corps de la requête.

Contrairement à une approche basée sur une session serveur, le frontend transmet directement les produits à acheter, ce qui rend l'API stateless (chaque requête contient toutes les informations nécessaires pour être traitée, sans dépendre d'une session côté serveur) et indépendante du stockage du panier côté backend.

Étapes détaillées :

1. Réception de la requête API

Le frontend envoie une requête POST vers (/api/payment) avec un payload JSON contenant :

- Les produits (id, quantité)
- Les informations de carte
- Les informations d'expédition

2. Validation des données d'entrée

L'API valide :

- La présence des produits
- Le format des informations de carte (numéro, nom, CVV, expiration, code postal)
- La présence des informations d'expédition

En cas d'erreur, une réponse JSON est retournée avec un statut "error".

3. Vérification des produits et du stock

Pour chaque produit :

- Vérification de l'existence en base
- Vérification du stock disponible

Si un produit est invalide ou en rupture :

- Le processus est interrompu
- Une erreur est retournée

4. Calcul du montant total

Le backend calcule dynamiquement :

- Le sous-total (prix × quantité)
- Les taxes
- Les frais additionnels (ex: écofrais)

Le montant total est ensuite déterminé côté serveur pour éviter toute manipulation côté client.

5. Validation des informations de paiement

Les informations de carte sont vérifiées dans la base de données via un modèle de validation.

Si la carte est invalide :

- Le paiement est refusé
- Une erreur est retournée

6. Début de la transaction (base de données)

Une transaction SQL est ouverte afin de garantir l'intégrité des données.

7. Vérification du client et du solde

Le système :

- Vérifie que le client existe
- Récupère le solde courant
- Vérifie que le solde est suffisant

Si le solde est insuffisant :

- La transaction est annulée
- Une erreur est retournée

8. Création des données métier

a. Création de l'expédition

Les informations d'expédition sont enregistrées en base.

b. Création des items

Chaque produit est enregistré comme item lié à l'expédition.

c. Mise à jour du stock

Le stock de chaque produit est décrémenté.

d. Création du paiement

Un enregistrement de paiement est créé avec :

- Le montant
- Le statut (success)
- Les informations de carte (partielles)

e. Transaction bancaire

Une transaction bancaire est enregistrée et le solde du client est mis à jour.

9. Validation finale

Si toutes les opérations réussissent :

- La transaction SQL est validée (commit)
- Une réponse JSON "success" est retournée

10. Gestion des erreurs

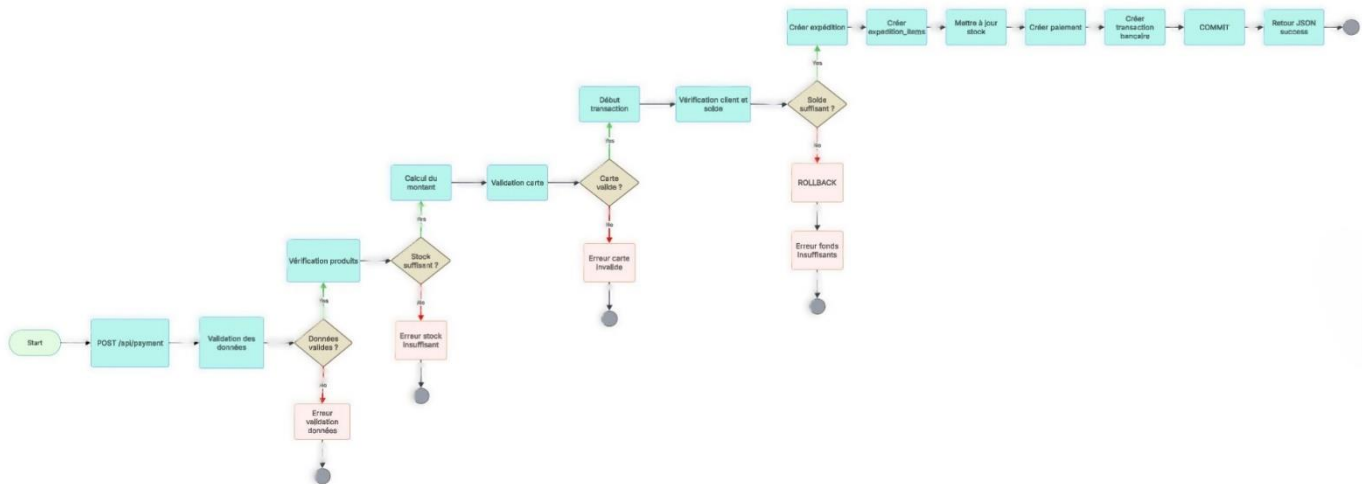
En cas d'exception :

- La transaction est annulée (rollback)
- Une réponse JSON "error" est retournée

Ce mécanisme garantit :

- La cohérence des données
- L'atomicité des opérations
- La sécurité du processus de paiement

Diagramme de Flux Détaillé du Paiement



Règles de gestion

Les règles de gestion définissent les conditions nécessaires au bon fonctionnement du système, en particulier pour le processus de paiement.

Règles générales :

- Toutes les requêtes API doivent être authentifiées
- Les données envoyées doivent être valides et complètes
- Les réponses API doivent être structurées en JSON

Règles liées au panier et aux produits :

- Un paiement ne peut être effectué que si au moins un produit est présent
- Chaque produit doit exister en base de données
- La quantité demandée doit être disponible en stock
- Le stock est décrémenté uniquement après validation du paiement

Règles liées au paiement :

- Les informations de paiement doivent être valides (format carte, CVV, expiration, etc.)
- Les informations de carte doivent correspondre à un enregistrement valide en base de données
- Le montant total doit être calculé côté serveur (et non côté client)
- Les taxes et frais doivent être appliqués automatiquement

Règles financières :

- Le client doit exister en base de données
- Le client doit disposer d'un solde suffisant pour effectuer le paiement
- Aucun paiement ne peut être validé si le solde est insuffisant
- Toute tentative de paiement sans fonds entraîne un échec immédiat

Règles de transaction :

- Toutes les opérations de paiement doivent être exécutées dans une transaction SQL
- En cas d'erreur, toutes les opérations doivent être annulées (rollback)
- En cas de succès, toutes les opérations doivent être validées (commit)
- Les données doivent rester cohérentes en toute circonstance

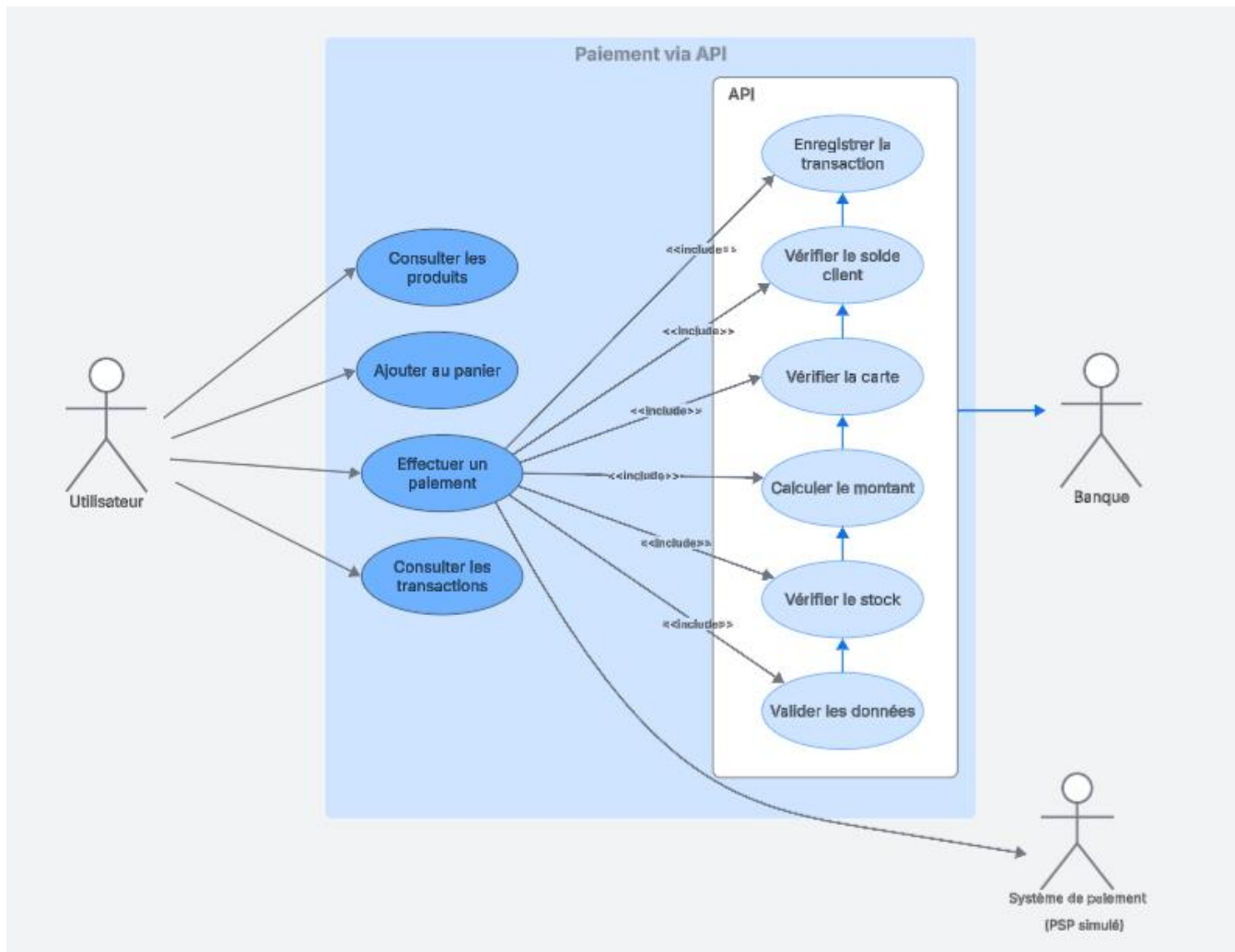
Règles de persistance des données :

- Une expédition doit être créée pour chaque paiement réussi
- Chaque produit acheté doit être enregistré comme item lié à l'expédition
- Un enregistrement de paiement doit être créé
- Une transaction bancaire doit être enregistrée
- Le solde du client doit être mis à jour après paiement

Règles de sécurité :

- Les entrées utilisateur doivent être validées côté serveur
- Les requêtes doivent être protégées contre les injections SQL
- Les données sensibles (ex : carte) ne doivent pas être stockées en clair
- L'accès aux endpoints critiques doit être contrôlé

Diagramme des cas d'utilisation - API de paiement



4. Description des cas d'utilisation

USE CASE #1	Authentification utilisateur	
Résumé	Permet à un utilisateur de s'authentifier afin d'accéder aux fonctionnalités de l'application, notamment le paiement, et de se déconnecter pour sécuriser sa session.	
Acteur principal	Utilisateur (client)	
Intervenants & Intérêts	Intervenants	Intérêts
	Utilisateur	Souhaite accéder à son compte et effectuer des opérations.
	Système	API Doit vérifier les identifiants et sécuriser la session.
Préconditions	Le compte utilisateur existe en base de données.	
Garanties de succès	- Session utilisateur active après connexion - Session correctement détruite après déconnexion	
Déclencheur	Envoi d'une requête de connexion (/api/login) ou de déconnexion (/api/logout).	
Scénario nominal - Connexion	Étape	Action
	1	L'utilisateur accède à la page de connexion.
	2	Il saisit ses identifiants (courriel, mot de passe).
	3	Le frontend envoie une requête à l'API (/api/login).
	4	Le système vérifie les informations en base de données.
	5	Le système crée une session utilisateur.
	6	Une réponse de succès est retournée avec authentification.
Scénario nominal - Déconnexion	Étape	Action
	1	L'utilisateur déclenche la déconnexion.
	2	Le frontend appelle l'API (/api/logout).
	3	Le système détruit la session active.
	4	Une confirmation est retournée.
Extensions	Étape	Action
	4a	Identifiants incorrects → réponse erreur (401).
	4b	Utilisateur inexistant → réponse erreur.

USE CASE #2	Paielement	
Résumé	Permet à un utilisateur d'effectuer un paiement en envoyant les données des produits, les informations de paiement et les informations d'expédition à l'API.	
Acteur principal	Utilisateur (client)	
Intervenants & Intérêts	Intervenants	Intérêts
	Utilisateur	Souhaite finaliser un achat
	Système	API Doit valider et traiter le paiement et Valide la transaction financière
Préconditions	L'utilisateur est authentifié.	
	Les produits à acheter sont disponibles.	
	Les informations de paiement et d'expédition sont fournies.	
Garanties de succès	Le paiement est validé.	
	Une expédition est créée.	
	Les produits sont enregistrés.	
	Une transaction bancaire est enregistrée.	
	Le solde du client est mis à jour.	
Déclencheur	Envoi d'une requête POST vers /api/payment.	
Scénario nominal	Étape	Action
	1	L'utilisateur envoie une requête avec produits, paiement et expédition.
	2	Le système valide les données reçues.
	3	Le système vérifie la disponibilité des produits.
	4	Le système calcule le montant total de la transaction.
	5	Le système valide les informations de carte
	6	Le système vérifie le solde du client
	7	Le système démarre une transaction SQL
	8	Le système crée l'expédition.
	9	Le système enregistre les produits (items).
	10	Le système met à jour le stock.
	11	Le système crée le paiement.
	12	Le système enregistre la transaction bancaire.
	13	Le système valide la transaction (commit).
	14	Le système retourne une réponse JSON de succès.
Extensions	Étape	Action
	2a	Données invalides → erreur retournée
	3a	Stock insuffisant → erreur
	5a	Carte invalide → paiement refusé
	6a	Solde insuffisant → paiement refusé
	7a	Erreur base de données → rollback

IV – Conception

Architecture de l'API

L'API est de type REST et utilise le protocole HTTP pour communiquer avec le frontend. Les données sont échangées au format JSON.

Chaque fonctionnalité est exposée via un endpoint spécifique utilisant les méthodes HTTP : GET, POST, PUT, DELETE.

Structure des endpoints

Utilisateurs		
Méthode	Endpoint	Description
POST	/api/login	Authentification
POST	/api/logout	Déconnexion

Panier		
Méthode	Endpoint	Description
GET	/api/cart	Voir le panier
POST	/api/cart	Ajoute un produit
DELETE	/api/cart/{id}	Supprimer un produit

Paieement		
Méthode	Endpoint	Description
POST	/api/payment	Effectuer un paiement

Transactions		
Méthode	Endpoint	Description
GET	/api/transactions	Liste des transaction
GET	/api/transactions/{id}	Détail d'une transaction

Endpoint BASEURL <http://etransactionnelle.atwebpages.com/>

```
◆ //LOGIN:
◆ $router->post('/api/login', 'LoginController@loginAPI');
◆
◆ //LOGGED-IN USER:
◆ $router->get('/api/currentuser', 'ClientManagementController@currentUser');
◆
◆ //PRODUCTS:
◆ $router->get('/api/products', 'ProductController@getProductsAPI');
◆
◆ //CART:
◆ $router->post('/api/cart/add', 'CartController@add');
◆ $router->get('/api/cart', 'CartController@get');
◆ $router->post('/api/cart/remove', 'CartController@remove');
◆ $router->post('/api/cart/update', 'CartController@update');
◆ $router->post('/api/cart/clear', 'CartController@clear');
◆
◆ //PAYMENT:
◆ $router->post('/api/payment', 'PaymentController@paymentAPI');
◆
◆ //Verification success {id}
◆ $router->get('/api/payment/details',
  'PaymentController@getPaymentDetailsAPI');
◆
◆ //EXPEDITION:
◆ $router->get('/api/expedition', 'ExpeditionController@expeditionsAPI');
◆
◆ //Expeditions Details {id}
◆ $router->get('/api/expedition/details',
  'ExpeditionController@expeditionDetailsAPI');
◆
◆ //LOGOUT:
◆ $router->post('/api/logout', 'LoginController@logoutAPI');
```

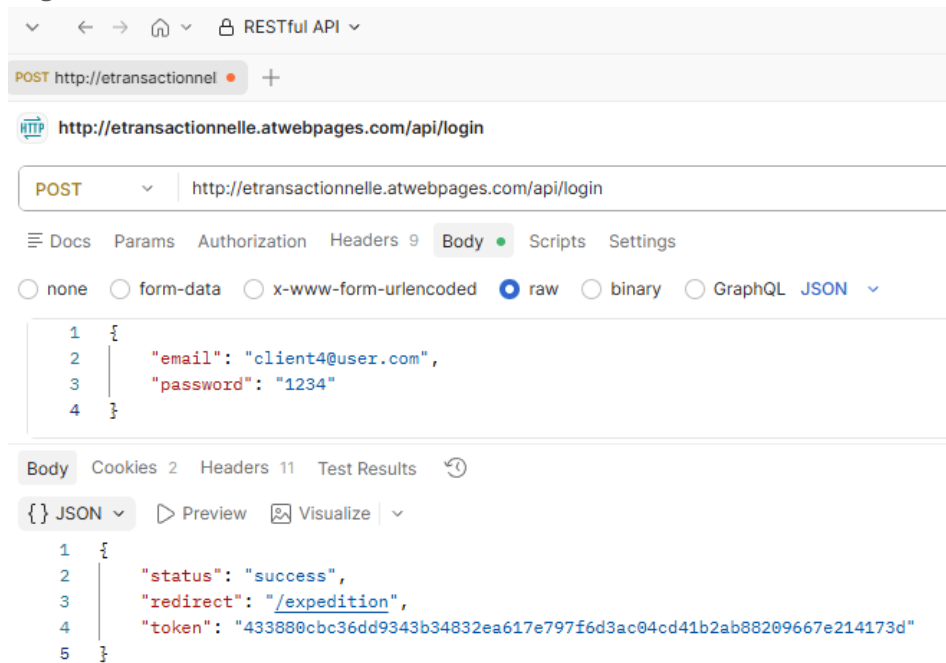
Headers http

Toutes les requêtes doivent contenir :

Content-Type: application/json

X-Auth-Token: API_TOKEN

Login :



Codes de réponse http

Code	Signification
200	Succès
201	Ressource créée
400	Requête invalide
401	Non autorisé
404	Ressource non trouvée
500	Erreur serveur

Sécurité de l'API

L'API intègre plusieurs mécanismes de sécurité afin de protéger les données et les opérations sensibles.

Authentification :

- ◆ Chaque requête protégée nécessite un token d'authentification
- ◆ Le token est généré lors de la connexion utilisateur
- ◆ Le token est stocké en base de données (table user_tokens)
- ◆ Le token est transmis automatiquement via un cookie sécurisé httpOnly (auth_token)
- ◆ Un header HTTP personnalisé X-Auth-Token peut également être utilisé comme mécanisme alternatif
- ◆ Le token est validé côté serveur à chaque requête protégée
- ◆ Une seule session active est autorisée par utilisateur :
 - Lors d'une nouvelle connexion, les anciens tokens sont supprimés
 - Une durée d'expiration est appliquée au token :
Exemple :
 - `date('Y-m-d H:i:s', strtotime('+1 hour'));`
 - La date d'expiration est enregistrée en base de données
 - La validation de l'expiration est effectuée côté serveur via PHP (strtotime() et time())
 - Lors de la déconnexion :
 - Le token est supprimé de la base de données
 - Le cookie auth_token est invalidé côté navigateur

Gestion de session et inactivité :

- ◆ Un mécanisme de déconnexion automatique est appliqué en cas d'inactivité utilisateur
 - L'activité utilisateur est surveillée côté client :
 - Clics
 - mouvements souris
 - clavier
 - scroll
 - interactions tactiles
- ◆ Après une période d'inactivité :
 - une requête /api/logout est exécutée
 - le token est supprimé
 - l'utilisateur est redirigé vers la page d'accueil / connexion

Exemple :

 - `const INACTIVITY_TIME = 15 * 60 * 1000;`
 - Une vérification d'authentification est également effectuée sur les pages protégées (authGuard.js) afin de bloquer l'accès si le token est expiré ou invalide.

Validation des entrées

- ◆ Toutes les données sont validées côté serveur
- ◆ Vérification des formats :
 - numéro de carte
 - CVV
 - date d'expiration
 - code postal
- ◆ Validation des champs obligatoires
- ◆ Validation des quantités et des données numériques

Protection des données sensibles :

- ◆ Les informations complètes de carte bancaire ne sont pas stockées
- ◆ Seules les données partielles (last4) sont conservées
- ◆ Les tokens d'authentification sont générés aléatoirement via :
 - `bin2hex(random_bytes(32));`
- ◆ Les cookies d'authentification utilisent les attributs :
 - ◆ httpOnly
 - ◆ SameSite=Lax

Sécurité des transactions :

- ◆ Utilisation de transactions SQL (commit / rollback)
- ◆ Prévention des incohérences en cas d'erreur
- ◆ Vérification du stock et du solde avant validation du paiement
- ◆ Vérification du solde avant traitement de la transaction
- ◆ Protection contre les paiements invalides ou incomplets

Contrôle des accès :

- ◆ Vérification de l'utilisateur avant toute opération critique
- ◆ Validation du token avant l'exécution des endpoints protégés
- ◆ Protection des endpoints sensibles :
 - /api/payment
 - /api/payment/details
 - /api/currentuser
 - /api/cart
 - /api/transactions
 - /api/logout
- ◆ Gestion des rôles :
 - Administrateur
 - Client
- ◆ Restriction des accès selon le rôle utilisateur (authMiddleware)

Diagrammes UML

Diagramme de classe

Représente la structure des entités (client, panier, transaction) et leurs relations.

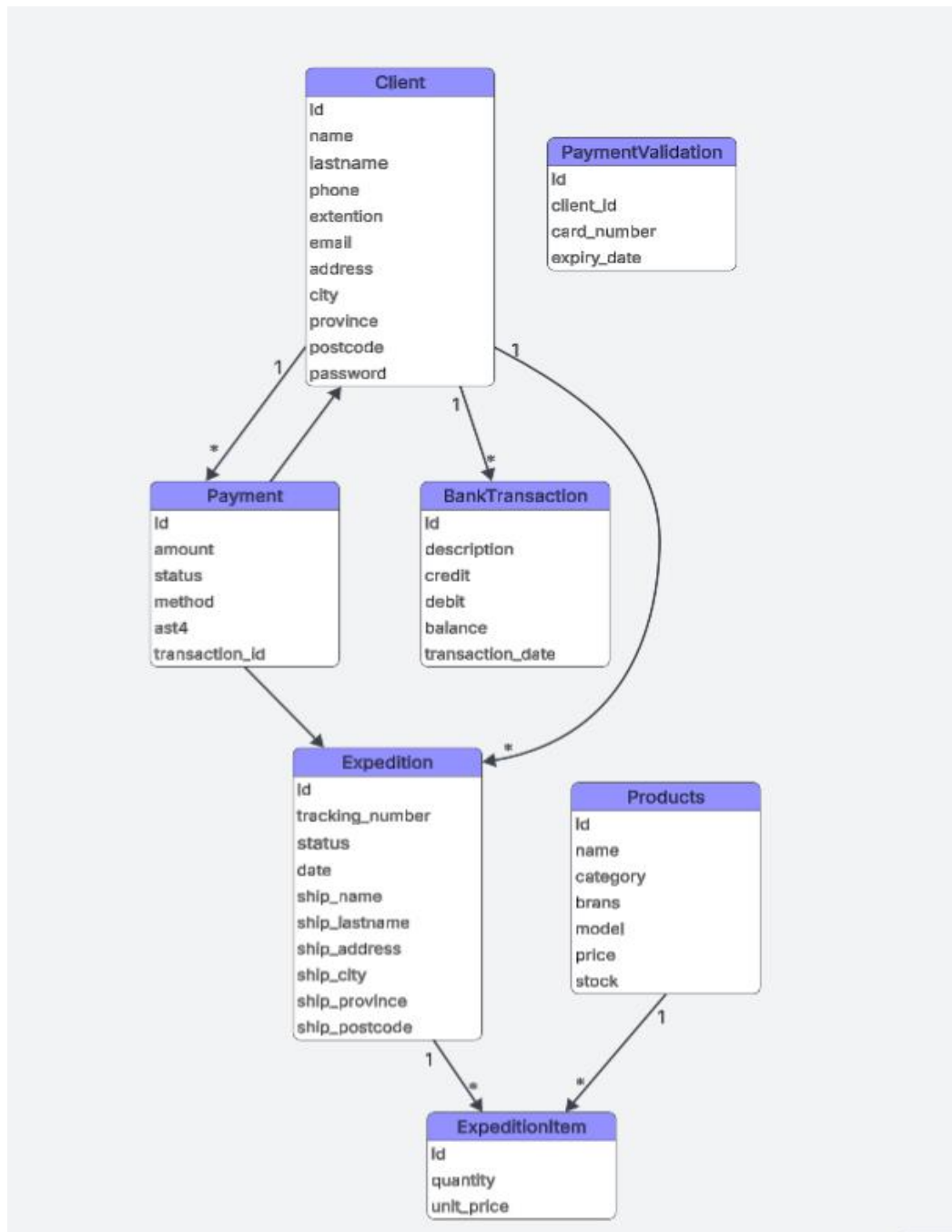
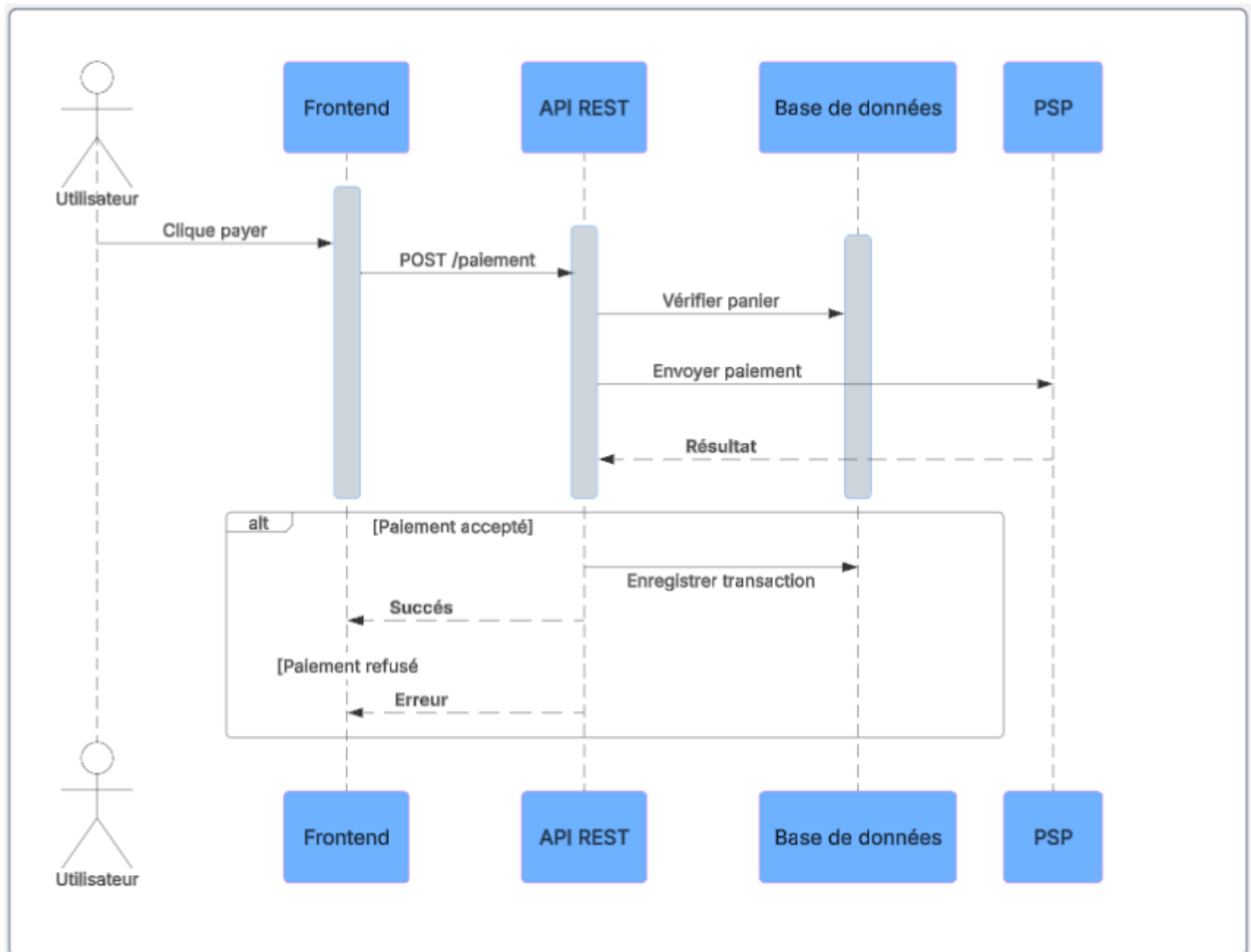


Diagramme de séquence

Illustre le processus d'un paiement entre le client, l'API et le système de paiement simulé.



Plan de test (Contient des cas de test)

Le présent plan de test a pour objectif de vérifier le bon fonctionnement, la fiabilité et la sécurité de l'API avant sa mise en production.

Les tests permettent de valider que les endpoints répondent correctement aux requêtes et respectent les spécifications définies.

In Scope	Out of Scope
Authentification (login, logout)	Interface utilisateur (frontend)
Utilisateurs	Intégration avec un PSP réel
Gestion du panier	Performance avancée
Panier	
Paielement	
Transactions	

Stratégie / approche de test

Les tests réalisés dans le cadre du projet visent à vérifier le bon fonctionnement de l'API dans des conditions réelles d'utilisation.

L'approche adoptée combine des tests fonctionnels, des tests d'intégration ainsi que des validations de sécurité afin de couvrir l'ensemble du flux transactionnel.

Les principaux tests effectués sont :

Tests fonctionnels des endpoints API :

- ◆ Vérification des routes GET, POST, PUT et DELETE afin de s'assurer que chaque endpoint retourne les réponses attendues et traite correctement les données reçues.

Tests d'intégration :

- ◆ Validation des échanges entre l'API, la base de données et le système de paiement simulé (PSP simulé).
Ces tests permettent de confirmer que les opérations critiques, comme le paiement et la mise à jour du stock, sont correctement synchronisées.

Tests de sécurité :

- ◆ Vérification du mécanisme d'authentification par token, du contrôle d'accès aux endpoints protégés et de la validation des données côté serveur.

Objectif des tests :

- ◆ Vérifier la stabilité et la fiabilité des endpoints API
- ◆ Assurer la cohérence des données enregistrées en base de données
- ◆ Valider les scénarios de succès ainsi que les cas d'erreur
- ◆ Garantir la sécurité des opérations sensibles, notamment le processus de paiement
- ◆ Confirmer le bon fonctionnement du flux transactionnel complet (panier → paiement → transaction)

Exemples de cas de test

Cas de Test : Authentification API

ID du test : CT-API-001

Préconditions :

1. L'API est déployée et accessible.
2. La base de données est active.
3. Un utilisateur valide existe dans la base de données.
4. L'utilisateur possède :
 - un courriel valide
 - un mot de passe valide
 - un compte actif

Étapes :

1. Ouvrir Postman ou l'outil de test API
2. Sélectionner la méthode HTTP POST
3. Entrer l'URL /api/login
4. Ajouter l'en-tête Content-Type: application/json
5. Envoyer les identifiants valides dans le body JSON
6. Envoyer la requête

Résultat attendu :

- ◆ 200 OK.

Validations à effectuer :

- ◆ Token API est présent.
- ◆ Le token n'est pas vide.
- ◆ Les informations utilisateur correspondent à la base de données.
- ◆ Aucun message d'erreur n'est retourné.
- ◆ La réponse est au format JSON valide.

Cas de Test : Authentification API invalide

ID du test : CT-API-002

Étapes :

1. Ouvrir Postman ou l'outil de test API
2. Sélectionner la méthode HTTP POST
3. Entrer l'URL /api/login
4. Ajouter l'en-tête Content-Type: application/json
5. Fournir un courriel valide
6. Fournir un mot de passe invalide
7. Envoyer la requête
8. Observer la réponse retournée par l'API

Résultat attendu :

- ◆ 401 Unauthorized

Validations à effectuer :

- ◆ La connexion doit être refusée.
- ◆ Aucun token ne doit être généré.
- ◆ Aucun accès utilisateur ne doit être accordé.
- ◆ Le message d'erreur doit être clair.
- ◆ La réponse doit être au format JSON valide.
- ◆ Le code HTTP doit être 401 Unauthorized.

Cas de Test : Paiement réussi

ID du test : CT-API-004

Préconditions :

- ◆ L'API est accessible.
- ◆ L'utilisateur est authentifié.
- ◆ Une expédition existante est disponible.
- ◆ Un montant valide est calculé pour la transaction.
- ◆ Une méthode de paiement valide est disponible (carte ou API carte).
- ◆ Les données de carte sont valides.

Étapes :

- ◆ Ouvrir Postman ou outil de test API
- ◆ Sélectionner la méthode HTTP POST
- ◆ Entrer l'URL /api/payment
- ◆ Ajouter l'en-tête Content-Type: application/json
- ◆ Ajouter les données de paiement valides

- ◆ Envoyer la requête
- ◆ Vérifier la réponse de l'API
- ◆ Vérifier en base de données que le paiement est enregistré

Résultat attendu :

- ◆ 200 ok

Validations à effectuer :

- ◆ Le paiement est accepté par l'API.
- ◆ Une transaction est générée.
- ◆ Les données sont enregistrées en base.
- ◆ Le statut du paiement est "success".
- ◆ Une réponse JSON valide est retournée.
- ◆ Aucun message d'erreur n'est présent.

Cas de Test : Paiement refusé

ID du test : CT-API-005

Étapes :

1. Envoyer POST /api/payment avec erreur (ex: solde insuffisant)
2. Observer la réponse retournée par l'API

Résultat attendu :

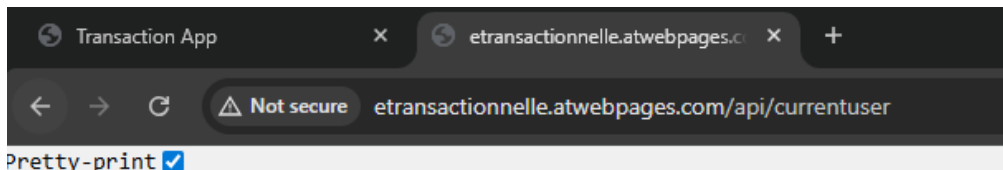
- ◆ Paiement refusé
- ◆ Réponse erreur

Validations à effectuer :

- ◆ Le paiement doit être refusé.
- ◆ Aucun débit ne doit être effectué.
- ◆ Aucune transaction ne doit être enregistrée comme réussie.
- ◆ Le message d'erreur doit être clair.
- ◆ La réponse doit être au format JSON valide.
- ◆ Le code HTTP doit correspondre à une erreur (ex: 400 ou 402 selon implémentation).

Endpoint testé :

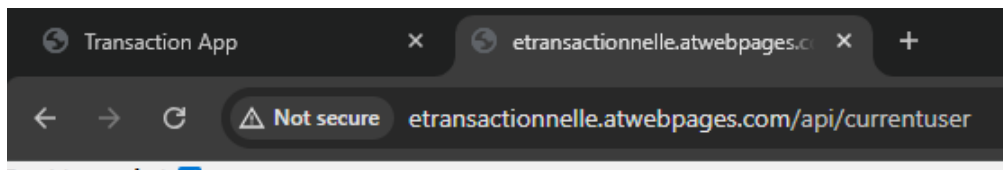
Validation de l'authentification utilisateur et génération du token d'accès.



The screenshot shows a web browser with two tabs: 'Transaction App' and 'etransactionnelle.atwebpages.com'. The address bar displays 'etransactionnelle.atwebpages.com/api/currentuser' with a 'Not secure' warning. Below the browser, a 'Pretty-print' checkbox is checked. The JSON response is as follows:

```
{
  "status": "success",
  "client": {
    "id": 761,
    "name": "Client",
    "lastname": "User",
    "email": "client5@user.com",
    "role": "client",
    "date": "2026-05-06 11:33:25"
  },
  "token": "2a956c526213240acb35dd8efd20e72a6540922fdcbcad32edf84c2786674ece"
}
```

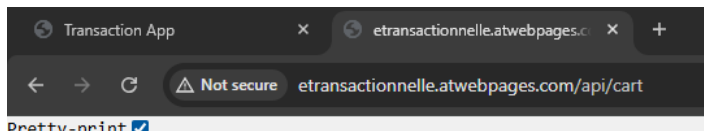
Authentication requise.



The screenshot shows the same web browser setup as before. The address bar still displays 'etransactionnelle.atwebpages.com/api/currentuser'. The 'Pretty-print' checkbox is checked. The JSON response is an error:

```
{
  "status": "error",
  "message": "Accès non autorisé"
}
```

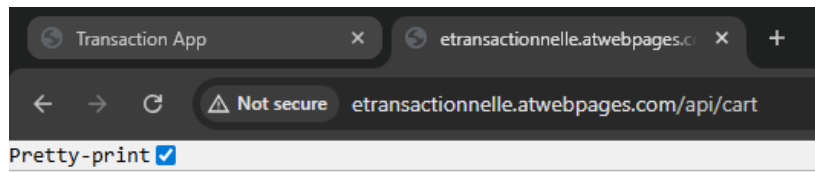
Produit ajouté au panier avec succès



The screenshot shows the web browser with the address bar displaying 'etransactionnelle.atwebpages.com/api/cart'. The 'Pretty-print' checkbox is checked. The JSON response is as follows:

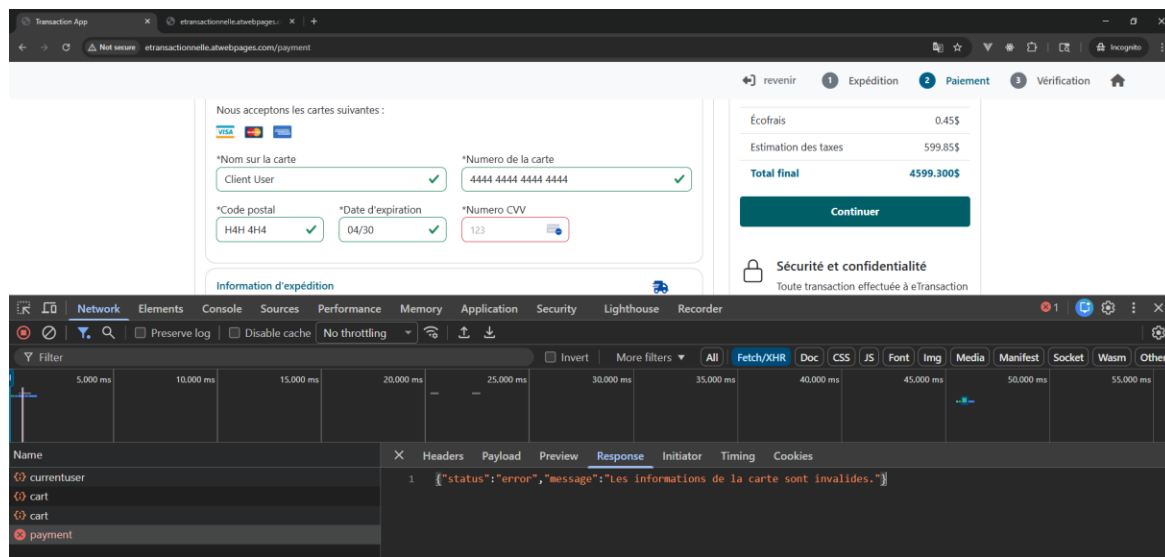
```
{
  "status": "success",
  "cart": [
    {
      "id": 7,
      "name": "Cisco UCS C220 M6",
      "price": "3999.00",
      "quantity": 1
    }
  ]
}
```

Aucun produit dans le panier.

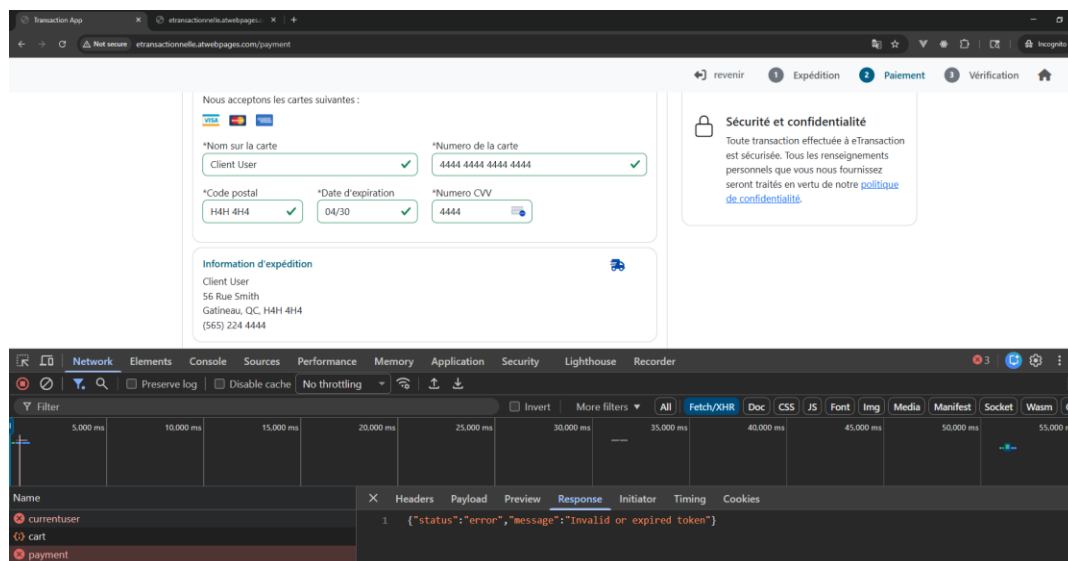


```
{
  "status": "success",
  "cart": []
}
```

Les informations de la carte sont invalides



Token invalide ou expiré



Le paiement a été effectué avec succès



```
{
  "status": "success",
  "payment": {
    "id": 100,
    "expedition_id": 132,
    "amount": "4599.30",
    "status": "success",
    "method": "API Carte",
    "last4": "4444",
    "transaction_id": "SIM69FB6F93CD6E3",
    "created_at": "2026-05-06 16:42:59",
    "updated_at": "2026-05-06 16:42:59",
    "client_id": 760
  },
  "client": {
    "id": 760,
    "name": "Client",
    "lastname": "User",
    "email": "client4@user.com",
    "phone": "(565) 224 4444",
    "extention": null,
    "address": "56 Rue Smith",
    "city": "Gatineau",
    "province": "QC",
    "postcode": "H4H 4H4",
    "password": "1234",
    "role": "client"
  },
  "expedition": {
    "id": 132,
    "client_id": 760,
    "ship_email": "client4@user.com",
    "ship_address": "56 Rue Smith",
    "ship_city": "Gatineau",
    "ship_province": "QC",
    "ship_postcode": "H4H 4H4",
    "date": "2026-05-06",
    "status": "pending",
    "tracking_number": "TRACK69FB6F93CCB6B",
    "ship_name": "Client",
    "ship_lastname": "User",
    "ship_phone": "(565) 224 4444",
    "name": "Client",
    "lastname": "User",
    "phone": "(565) 224 4444"
  },
  "commands": [
    {
      "quantity": 1,
      "price": "3999.00",
      "product_name": "Cisco UCS C220 M6"
    }
  ]
}
```

V – Planification

[illegible]

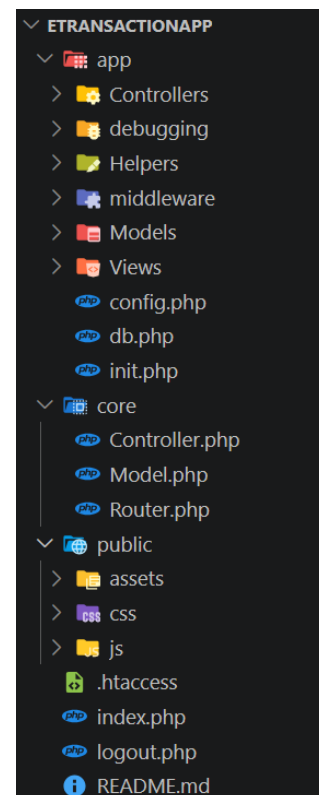
VI – Développement

L'application est organisée selon une architecture MVC (Model – View – Controller) permettant de séparer la logique métier, l'accès aux données et l'interface utilisateur.

Structure du projet :

Cette structure facilite :

- ◆ La maintenance du projet
- ◆ La réutilisation du code
- ◆ L'évolution de l'application
- ◆ La séparation des responsabilités



Description des composants

Controllers

Contient les contrôleurs qui gèrent les requêtes utilisateur et appellent les modèles.

Exemples :

- ◆ LoginController.php → authentification
- ◆ ClientManagementController.php → gestion des clients
- ◆ ProductController.php → gestion des produits
- ◆ PaymentController.php → traitement des paiements
- ◆ ExpeditionController.php → traitement des expéditions

Models

Contient les classes qui interagissent avec la base de données.

Exemples :

- ◆ Client.php → gestion des utilisateurs
- ◆ Products.php → gestion des produits
- ◆ Payment.php → gestion des paiements
- ◆ BankTransaction.php → gestion des transactions
- ◆ Expedition.php → gestion des expéditions
- ◆ UserToken.php → gestion des tokens d'authentification (validation et contrôle d'accès API)

Views

Interfaces utilisateur (HTML + Bootstrap).

Exemples :

- ◆ auth/ → connexion
- ◆ clients/ → clients
- ◆ products/ → affichage produits
- ◆ payment/ → paiement
- ◆ expedition → expéditions

Public

Point d'entrée de l'application :

- ◆ index.php → point d'accès principal
- ◆ .htaccess → redirection vers le routeur
- ◆ logout.php → gestion de la déconnexion

Intégration de l'API dans MVC

L'API est directement intégrée dans l'architecture MVC de l'application.

Les contrôleurs jouent un rôle central en exposant des endpoints REST et en retournant des réponses au format JSON.

Contrairement à une application MVC classique orientée vues, les contrôleurs sont ici utilisés pour gérer simultanément :

- a. Les requêtes web (HTML)
- b. Les requêtes API (JSON)

Fonctionnement

Exemples de contrôleurs API : **PaymentController**

1. Le frontend envoie une requête HTTP (fetch / AJAX) vers un endpoint API

```
public > js > payment > JS paymentAPI.js > ...
1  export async function createPayment(data) {
2    const res = await fetch("/api/payment", {
3      method: "POST",
4      headers: {
5        "Content-Type": "application/json",
6      },
7      body: JSON.stringify(data),
8    });
9
10   return res.json();
11 }
12
13 export async function getCart() {
14   const res = await fetch(`${BASE}/api/cart`);
15   return res.json();
16 }

//=====
//-----PAYMENT-----
//=====
$router->post('/api/payment', 'PaymentController@paymentAPI');
$router->get('/api/payment', 'PaymentController@paymentAPI');
```

2. Le routeur (index.php + .htaccess) redirige la requête vers le contrôleur approprié

```
app > Controllers > PaymentController.php > PaymentController > paymentAPI()
25  class PaymentController
262  public function paymentAPI()
360      //API - Start transaction -----
361      $db = Database::getConnection();
362
363      try {
364          $db->beginTransaction();
365
366          //Get client id
367          $clientId = $_REQUEST['user']['id'];
368
369          // Check client exists
370          $clientModel = new Client();
371          $client = $clientModel->findById($clientId);
372
373          if (!$client) {
374              throw new \Exception("Client introuvable.");
375          }
376
377          // API - Check balance
378          $transactionModel = new BankTransaction($db);
379          $transactions = $transactionModel->getByClientId($clientId);
380
381          // API - Take latest balance
382          $currentBalance = $transactions[0]['balance'] ?? 0;
383
384          if ($currentBalance < $totalAmount) {
385              throw new \Exception("Fonds insuffisants.");
386          }
387
388
389          // API - Get expedition data from request
390          $expeditionModel = new Expedition();
391
392          $expeditionData = $data['expedition'] ?? [];
393
394          if (empty($expeditionData)) {
395              $this->json([
396                  'status' => 'error',
397                  'message' => 'Expedition data required'
398              ], 400);
399          }
400      }
```

3. Le contrôleur :

- ◆ Valide la requête
- ◆ Appelle les modèles nécessaires
- ◆ Applique la logique métier
- ◆ Le contrôleur retourne une réponse JSON

4. Les modèles interagissent avec la base de données

```
app > Models > Payment.php > ...
14 class Payment extends Model
88 public function findById($id)
89 {
90     $sql = "SELECT * FROM {$this->table} WHERE id = :id LIMIT 1";
91     $stmt = $this->db->prepare($sql);
92     $stmt->execute([':id' => $id]);
93     return $stmt->fetch(PDO::FETCH_ASSOC);
94 }
95
96 }
97
```

PaymentController :

- Expose l'endpoint /api/payment
- Traite le paiement complet (validation, transaction, stockage)
- Retourne une réponse JSON (success / error)

Gestion de l'authentification

Les endpoints sensibles sont protégés via un middleware (apiAuth) qui :

- Vérifie la présence du token (Authorization: Bearer)
- Contrôle l'accès à l'API
- Bloque les requêtes non autorisées

```
app > middleware > apiAuth.php > ...
1 <?php
2
3 use App\Models\UserToken;
4
5 7 references
6 function apiAuth()
7 {
8     header('Content-Type: application/json');
9
10    // Get headers
11    $headers = getallheaders();
12
13    // Try custom header first
14    $token = $headers['X-Auth-Token'] ?? $headers['x-auth-token'] ?? null;
15
16    // Fallback to httpOnly cookie
17    if (!$token && isset($_COOKIE['auth_token'])) {
18        $token = $_COOKIE['auth_token'];
19    }
20
21    // No token found
22    if (!$token) {
23        http_response_code(401);
24        echo json_encode([
25            'status' => 'error',
26            'message' => 'Unauthorized access: No token provided'
27        ]);
28        exit;
29    }
30 }
```

Séparation Web / API

L'application distingue deux types de réponses :

- HTML → pour les pages (views)
- JSON → pour les endpoints API

Cette approche permet :

- ◆ Une réutilisation des contrôleurs
- ◆ Une séparation claire entre interface utilisateur et logique API
- ◆ Une évolution vers une architecture plus découplée

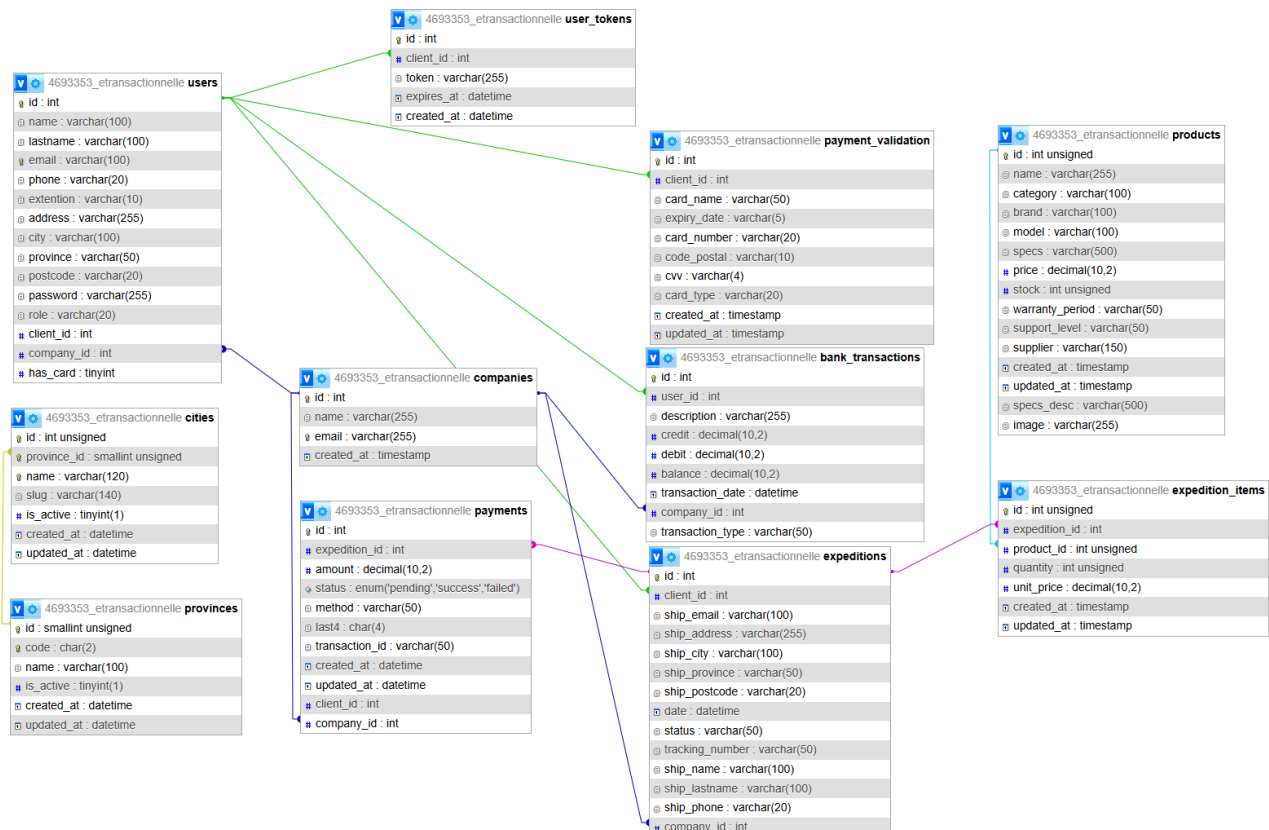
Structure de la base de données

La base de données s'appuie sur un modèle relationnel permettant de gérer les utilisateurs, les produits, les expéditions, les paiements et les transactions bancaires.

Le schéma est organisé autour du processus de paiement et de gestion des commandes.

- ◆ Un client effectue un paiement
- ◆ Un paiement génère une expédition
- ◆ Une expédition contient plusieurs produits (via expedition_items)
- ◆ Une transaction bancaire met à jour le solde du client

Cette structure garantit la cohérence des données et permet de tracer toutes les opérations financières.



Tables essentielles :

- ◆ **clients**
Contient les informations des utilisateurs : nom, email, adresse, téléphone, mot de passe.
- ◆ **products**
Contient les produits disponibles : nom, catégorie, prix, stock.
- ◆ **expeditions**
Représente les commandes (livraisons) associées à un paiement.
- ◆ **expedition_items**
Table de liaison entre expéditions et produits, avec quantité et prix unitaire.
- ◆ **payments**
Contient les informations des paiements effectués (montant, statut, méthode, transaction_id).
- ◆ **bank_transactions**
Gère les opérations financières et le solde des clients (débit, crédit, balance).
- ◆ **payment_validation**
Stocke les informations nécessaires à la validation des cartes (simulation).
- ◆ **user_tokens**
Gère les tokens d'authentification pour sécuriser l'accès à l'API.

Relations entre les tables résumées

Relation	Type	Description
client → payments	1 → *	Un client peut effectuer plusieurs paiements
client → bank_transactions	1 → *	Un client possède plusieurs transactions bancaires
client → expeditions	1 → *	Un client peut avoir plusieurs expéditions
expeditions → expedition_items	1 → *	Une expédition contient plusieurs produits
products → expedition_items	1 → *	Un produit peut apparaître dans plusieurs expéditions
payments → expeditions	1	Un paiement est associé à une expédition
client → user_tokens	1 → *	Un client peut avoir plusieurs tokens
client → payment_validation	1 → *	Un client peut avoir plusieurs méthodes de paiement enregistrées

SQL Structure

```
-- CLIENTS
-- =====
CREATE TABLE clients (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  lastname VARCHAR(100),
  email VARCHAR(100),
  phone VARCHAR(20),
  extention VARCHAR(10),
  address VARCHAR(255),
  city VARCHAR(100),
  province VARCHAR(50),
  postcode VARCHAR(20),
  password VARCHAR(255),
  role VARCHAR(20)
);

-- PRODUCTS
-- =====
CREATE TABLE products (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  category VARCHAR(100),
  brand VARCHAR(100),
  model VARCHAR(100),
  price DECIMAL(10,2),
  stock INT UNSIGNED
);

-- EXPEDITIONS
-- =====
CREATE TABLE expeditions (
  id INT AUTO_INCREMENT PRIMARY KEY,
  client_id INT,
  ship_name VARCHAR(100),
  ship_lastname VARCHAR(100),
  ship_address VARCHAR(255),
  ship_city VARCHAR(100),
  ship_province VARCHAR(50),
  ship_postcode VARCHAR(20),
  tracking_number VARCHAR(50),
  status VARCHAR(50),
  date DATE,
  FOREIGN KEY (client_id) REFERENCES clients(id)
);
```

```

-- EXPEDITION ITEMS
-- =====
CREATE TABLE expedition_items (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  expedition_id INT,
  product_id INT UNSIGNED,
  quantity INT UNSIGNED,
  unit_price DECIMAL(10,2),
  FOREIGN KEY (expedition_id) REFERENCES expeditions(id),
  FOREIGN KEY (product_id) REFERENCES products(id)
);

-- PAYMENTS
-- =====
CREATE TABLE payments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  client_id INT,
  expedition_id INT,
  amount DECIMAL(10,2),
  status VARCHAR(50),
  method VARCHAR(50),
  last4 CHAR(4),
  transaction_id VARCHAR(50),
  FOREIGN KEY (client_id) REFERENCES clients(id),
  FOREIGN KEY (expedition_id) REFERENCES expeditions(id)
);

-- BANK TRANSACTIONS
-- =====
CREATE TABLE bank_transactions (
  id INT AUTO_INCREMENT PRIMARY KEY,
  client_id INT,
  description VARCHAR(255),
  credit DECIMAL(10,2),
  debit DECIMAL(10,2),
  balance DECIMAL(10,2),
  transaction_date DATETIME,
  FOREIGN KEY (client_id) REFERENCES clients(id)
);

```

```

-- PAYMENT VALIDATION
-- =====
CREATE TABLE payment_validation (
  id INT AUTO_INCREMENT PRIMARY KEY,
  client_id INT,
  card_name VARCHAR(50),
  card_number VARCHAR(20),
  expiry_date VARCHAR(5),
  cvv VARCHAR(4),
  card_type VARCHAR(20),
  FOREIGN KEY (client_id) REFERENCES clients(id)
);

-- USER TOKENS
-- =====
CREATE TABLE user_tokens (
  id INT AUTO_INCREMENT PRIMARY KEY,
  client_id INT,
  token VARCHAR(255),
  expires_at DATETIME,
  FOREIGN KEY (client_id) REFERENCES clients(id)
);

```

Note :

Dans un environnement réel, les données sensibles telles que le CVV ne devraient jamais être stockées conformément aux exigences PCI-DSS.

Cette table est utilisée uniquement dans le cadre d'une simulation académique du système de paiement.

VII – Déploiement

Plan de déploiement en production

Le déploiement de l'API est réalisé sur un hébergement mutualisé AwardSpace.

Il suit un processus structuré permettant d'assurer la disponibilité, la sécurité et le bon fonctionnement du système.

Préparation de l'environnement :

- ◆ Création de la base de données MySQL via l'interface AwardSpace
- ◆ Configuration des accès (nom de la base, utilisateur, mot de passe)
- ◆ Définition des variables de connexion :
 - DB_HOST
 - DB_NAME
 - DB_USER
 - DB_PASS
- Vérification de la version PHP compatible (PHP 8.3)

Transfert des fichiers :

- ◆ Utilisation de FileZilla (SFTP) pour le déploiement
- ◆ Téléversement des fichiers du projet vers le serveur
- ◆ Vérification que le dossier /public est accessible (point d'entrée)
- ◆ Vérification des permissions (lecture/exécution)

Configuration de l'API

- ◆ Mise à jour des paramètres de connexion à la base de données
- ◆ Configuration du routage (.htaccess) pour rediriger vers index.php
- ◆ Vérification des endpoints API :
 - ◆ /api/login
 - ◆ /api/cart
 - ◆ /api/payment
 - ◆ /api/transactions
- ◆ Validation du format des réponses JSON

Tests en production

Tests réalisés via Postman ou requêtes HTTP :

- ◆ Authentification
- ◆ Gestion du panier
- ◆ Paiement (simulation PSP)
- ◆ Transactions bancaires

Objectifs :

- ◆ Vérifier les réponses http
- ◆ Valider les cas de succès et d'erreur
- ◆ Assurer la cohérence des données

Spécifications de l'hébergement AwardSpace

- ◆ Type : Hébergement mutualisé
- ◆ Serveur : Apache avec support PHP
- ◆ Version PHP : 8.3.26
- ◆ Base de données : MySQL 8.0
- ◆ Accès : SFTP (FileZilla)
- ◆ Domaine : <http://etransactionnelle.atwebpages.com/>

Sécurité en production

L'API intègre plusieurs mécanismes de sécurité :

- ◆ Authentification via token (Authorization: Bearer)
- ◆ Validation des données côté serveur
- ◆ Protection contre les injections SQL (requêtes préparées)
- ◆ Protection contre les attaques XSS (validation et filtrage)

Bonnes pratiques appliquées :

- ◆ Aucun stockage des données sensibles complètes (carte bancaire)
- ◆ Utilisation de transactions SQL pour garantir l'intégrité
- ◆ Contrôle des accès aux endpoints sensibles

Vérification post-déploiement

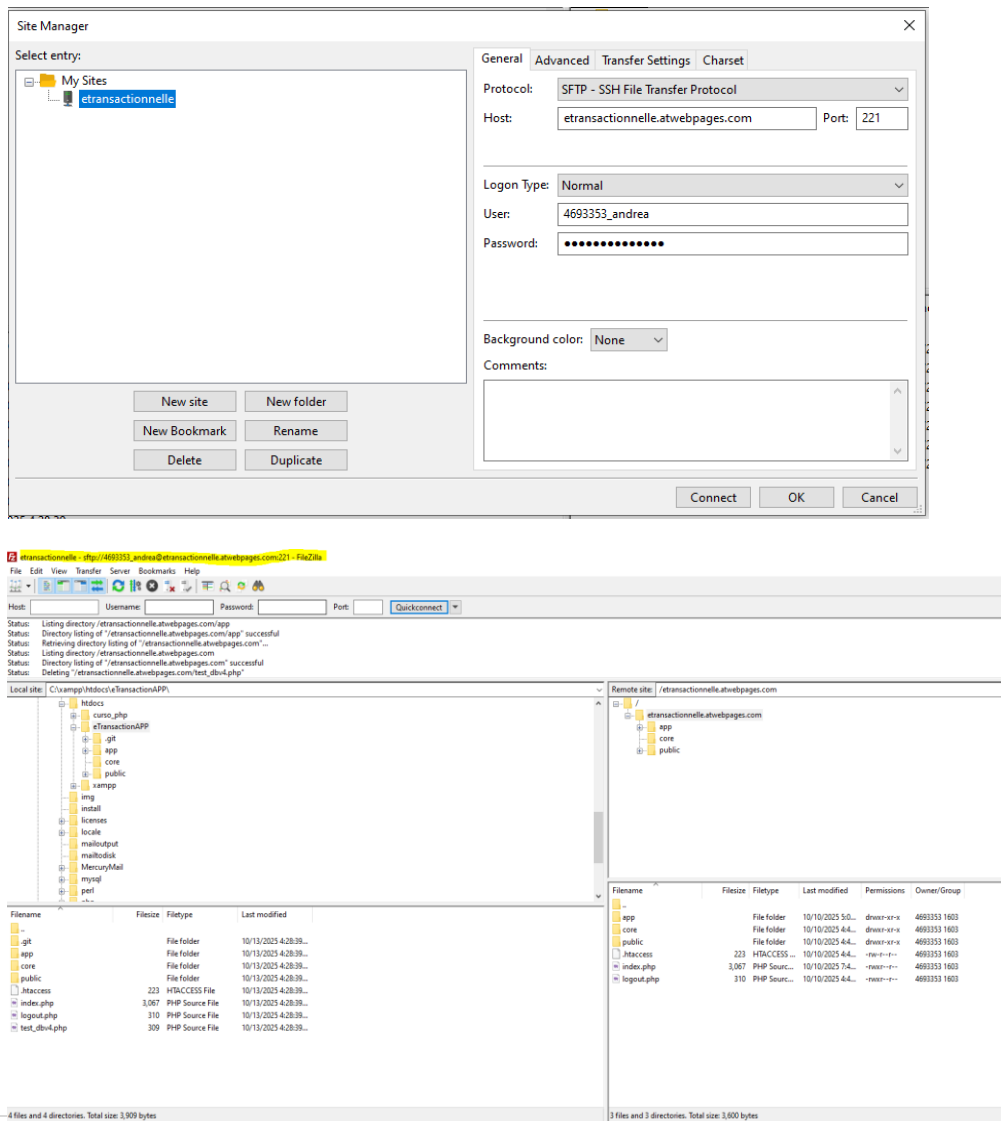
Après déploiement, les éléments suivants sont vérifiés :

- ◆ Accessibilité des endpoints API
- ◆ Connexion à la base de données
- ◆ Fonctionnement du paiement complet
- ◆ Mise à jour du stock et du solde client
- ◆ Gestion correcte des erreurs

Cette phase garantit que l'application est opérationnelle et stable en production.

Spécifications de l'hébergement

Transférer des fichiers :



Gestion des fichiers dans AwardSpace :

cp1.awardspace.net/file-manager/www.etransactionnelle.atwebpages.com

AWARDSPACE

Welcome, Andrea Silva

Dashboard

Hosting Tools

Buy Services

Cloud Servers

Account

Support

File Manager

etransactionnelle.atwebpages.com

app

core

public

Are you new to our File Manager?

Discover all built in features.

Open Tutorial

Back

Upload

Create

Rename

Move To

Delete

Extract

Download

Open

More...

/home/www.etransactionnelle.atwebpages.com

Go

Up Level

Home

Refresh

Select Multiple

Select All

Deselect All

Clipboard...

127 files, 592 KIB total size

Name	Size	Type	Date Modified	Permissions
/	-	Current	Oct 13 2025 21:34:07	drwxr-xr-x (755)
./	-	Parent	Oct 11 2025 0:45:46	dr-xr-xr-x (555)
app/	-	Directory	Oct 10 2025 21:07:14	drwxr-xr-x (755)
core/	-	Directory	Oct 10 2025 20:44:29	drwxr-xr-x (755)
public/	-	Directory	Oct 10 2025 20:40:57	drwxr-xr-x (755)
.htaccess	223 B	Web configuration	Oct 10 2025 20:40:01	-rw-r--r- (644)
index.php*	3 KIB	PHP script	Oct 10 2025 23:49:50	-rwxr--r- (744)
logout.php*	310 B	PHP script	Oct 10 2025 20:40:01	-rwxr--r- (744)

Total: 3 directories, 3 files, Total file size: 3.52 KIB

Copyright 2003 - 2025 - All rights reserved

Privacy Policy

Terms of Services

Refund Policy

Provisioning & Invoicing

Informations sur la base de données dans AwardSpace :

All Databases

MySQL Databases

PostgreSQL Databases

Search filter:

Type database to filter

Name	User	Host	Port	Quota	Management	Type	Options
4693353_etransactionnelle	4693353_etransactionnelle	pdb1042.awardspace.net	3306	Available: 250 MiB Used: 880 KIB	phpMyAdmin See all tools	MySQL	

Information

Password

Management

Delete

Database details for 4693353_etransactionnelle

Below you can find detailed information for your database.

Database Host:

pdb1042.awardspace.net

Database Port:

3306

Database Name:

4693353_etransactionnelle

Database User:

4693353_etransactionnelle

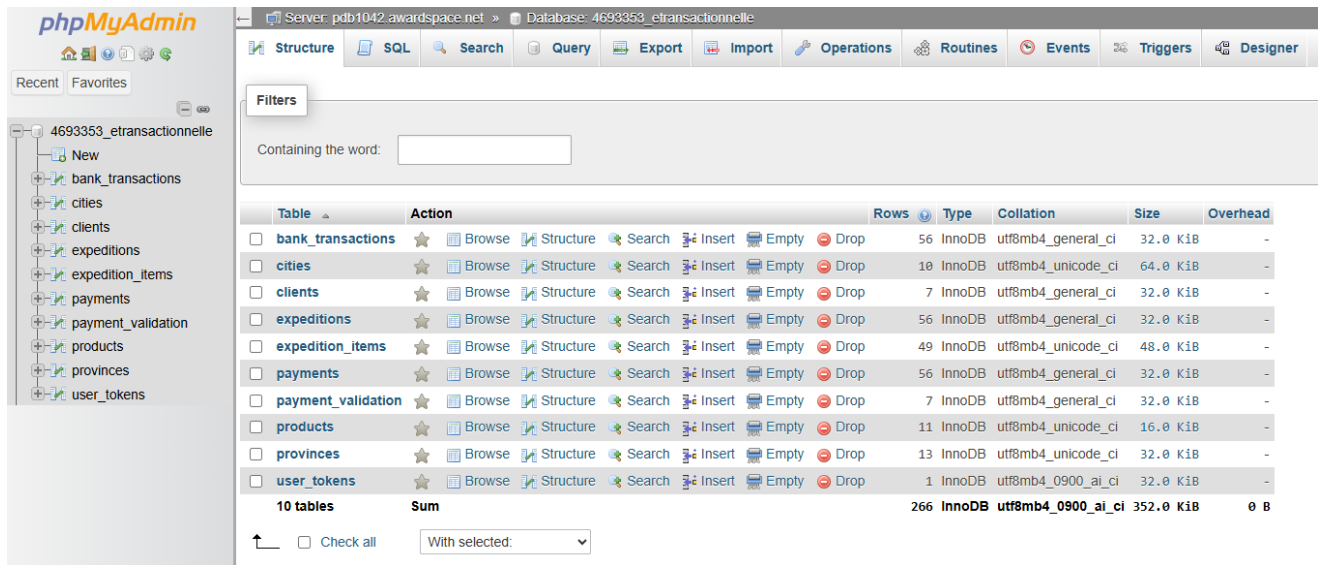
Database Password:

(The password for 4693353_etransactionnelle) [change](#)

Database Version:

8.0

Base de données phpMyAdmin AwardSpace :



The screenshot displays the phpMyAdmin interface for the database '4693353_etransactionnelle'. The left sidebar shows a tree view of the database structure with tables like 'bank_transactions', 'cities', 'clients', 'expeditions', 'expedition_items', 'payments', 'payment_validation', 'products', 'provinces', and 'user_tokens'. The main panel shows the 'Structure' tab with a list of tables and their properties.

Table	Action	Rows	Type	Collation	Size	Overhead
☐ bank_transactions	★ Browse Structure Search Insert Empty Drop	56	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ cities	★ Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_unicode_ci	64.0 KiB	-
☐ clients	★ Browse Structure Search Insert Empty Drop	7	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ expeditions	★ Browse Structure Search Insert Empty Drop	56	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ expedition_items	★ Browse Structure Search Insert Empty Drop	49	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ payments	★ Browse Structure Search Insert Empty Drop	56	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ payment_validation	★ Browse Structure Search Insert Empty Drop	7	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ products	★ Browse Structure Search Insert Empty Drop	11	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ provinces	★ Browse Structure Search Insert Empty Drop	13	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ user_tokens	★ Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_0900_ai_ci	32.0 KiB	-
10 tables Sum		266	InnoDB	utf8mb4_0900_ai_ci	352.0 KiB	0 B

Filters: Containing the word:

Check all With selected:

Conclusion

Ce projet a permis de concevoir et de développer une API REST transactionnelle complète intégrant l'authentification, la gestion du panier, le traitement des paiements et la gestion des transactions.

L'architecture mise en place favorise la séparation des responsabilités, la sécurité des opérations et l'évolutivité de l'application.

Les mécanismes de validation, de contrôle d'accès et de gestion transactionnelle assurent la cohérence, la fiabilité et la sécurité du système.

Cette approche permet également une évolution future vers une architecture plus modulaire et orientée services.